

# EduStruct — Memoria Técnica del Proyecto

## Programación III

Visualizador Interactivo de Estructuras de Datos Aplicadas al Dominio Educativo

## Información General

| Campo                       | Información                    |
|-----------------------------|--------------------------------|
| Proyecto                    | EduStruct                      |
| Curso                       | Programación III               |
| Tipo                        | Proyecto Académico             |
| Arquitectura                | Frontend + Backend desacoplado |
| Frontend                    | React + Vite                   |
| Backend                     | Flask                          |
| Visualización               | React Flow                     |
| Lenguaje principal backend  | Python                         |
| Lenguaje principal frontend | JavaScript                     |

## Tabla de Contenidos

- 1. Introducción
- 2. Objetivos
- 3. Alcance del Sistema
- 4. Arquitectura General
- 5. Stack Tecnológico
- 6. Organización del Proyecto
- 7. Modelo del Dominio Educativo
- 8. Implementación de Estructuras
- 9. APIs REST
- 10. Visualización Interactiva
- 11. Complejidad Algorítmica
- 12. Dockerización y Entorno
- 13. Resultados Obtenidos
- 14. Conclusiones
- 15. Trabajo Futuro
- 16. Anexos

---

# 1. Introducción

---

EduStruct es una plataforma académica desarrollada para visualizar estructuras de datos clásicas mediante un enfoque interactivo y contextualizado al dominio universitario.

El proyecto busca demostrar la implementación manual de estructuras fundamentales utilizando algoritmos reales, visualización gráfica y separación arquitectónica entre frontend y backend.

A diferencia de simuladores estáticos o ejemplos aislados, EduStruct integra:

- Árboles jerárquicos académicos
- Índices balanceados
- Grafos de prerrequisitos
- Tablas hash con colisiones
- Estructuras lineales contextualizadas
- Recorridos visuales
- Métricas estructurales

Todo ello mediante una arquitectura moderna basada en Flask, React y React Flow.

---

## 2. Objetivos

---

### 2.1 Objetivo General

Diseñar e implementar una plataforma interactiva para demostrar el funcionamiento interno de estructuras de datos clásicas aplicadas a un contexto educativo universitario.

---

### 2.2 Objetivos Específicos

- Implementar estructuras de datos manualmente sin depender de estructuras internas del lenguaje.
  - Visualizar recorridos y transformaciones estructurales mediante React Flow.
  - Simular escenarios académicos reales utilizando estructuras lineales y no lineales.
  - Aplicar algoritmos de búsqueda, recorrido y balanceo.
  - Construir una arquitectura desacoplada frontend/backend.
  - Documentar complejidad algorítmica y comportamiento estructural.
- 

## 3. Alcance del Sistema

---

### Incluye

- Árbol General
- Árbol Binario BST
- Árbol AVL

- Árbol B
  - Tabla Hash
  - Grafos DFS/BFS
  - Lista enlazada
  - Cola
  - Pila
  - Visualización interactiva
  - Métricas estructurales
  - APIs REST
  - Dockerización base
- 

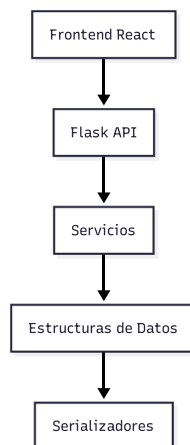
## No Incluye

- Persistencia permanente en base de datos
  - Sistema de autenticación
  - Gestión multiusuario
  - Despliegue cloud productivo
  - Seguridad avanzada
  - Integración CI/CD
- 

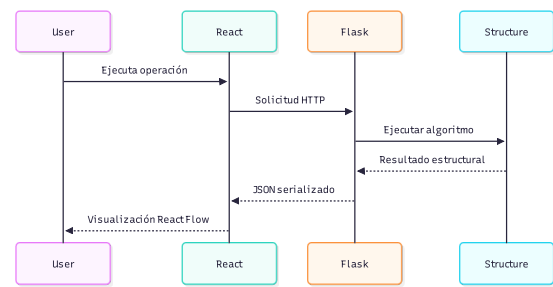
## 4. Arquitectura General

---

### Arquitectura lógica



### Flujo general del sistema



## 5. Stack Tecnológico

| Capa          | Tecnología   | Función                  |
|---------------|--------------|--------------------------|
| Frontend      | React + Vite | Interfaz visual          |
| Backend       | Flask        | API REST                 |
| Visualización | React Flow   | Diagramación interactiva |
| Estilos       | Tailwind CSS | Diseño visual            |
| Comunicación  | Axios        | Consumo API              |
| Contenedores  | Docker       | Entorno desacoplado      |

## 6. Organización del Proyecto

```
/backend
  /app
    /routes
    /services
    /structures
    /schemas
    /serializers

/frontend
  /src
    /pages
    /components
    /services
    /layouts

/docs
/datasets
```

## 7. Modelo del Dominio Educativo

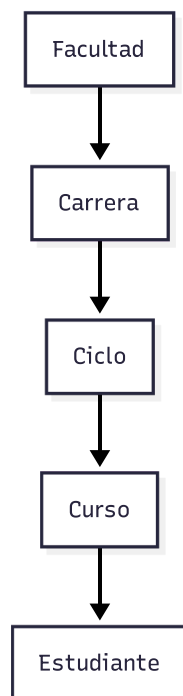
EduStruct contextualiza las estructuras de datos dentro de escenarios universitarios reales.

## Entidades principales

- Facultad
- Carrera
- Ciclo
- Curso
- Estudiante
- Expediente
- Prerrequisito
- Historial académico

---

## Relación académica



---

## 8. Implementación de Estructuras

---

### 8.1 Árbol General

---

#### Objetivo

Representar el pensum académico mediante una jerarquía de múltiples hijos.

#### Características implementadas

- Inserción de nodos
- Eliminación de subárboles

- Recorridos
- Métricas estructurales
- Visualización React Flow

Evidencia visual

EduStruct

Visualizador Interactivo de Estructuras de Datos

Inicio

Árbol General

Árbol Binario

AVL

Árbol B

Hash

Grafos

Estructuras Lineales

ÉPICA 5

Árbol General del Pensum Académico

Visualiza y manipula la jerarquía Facultad → Carrera → Ciclos → Cursos usando Flask, Python puro y React Flow.

Cargar demo

Reiniciar

Estado del árbol general obtenido correctamente.

NODOS

8

ALTURA

3

NIVELES

4

HOJAS

4

MÁX. HIJOS

2

ARISTAS

7

Insertar nodo

Usa parentid para colgar el nodo de un padre existente. Déjalo vacío solo cuando el árbol esté vacío y vayas a crear la raíz.

ID, ej. programming-2

Etiqueta, ej. Programación II

ID padre, ej. cycle-2

Curso

Código opcional, ej. SIS-202

Insertar

Eliminar nodo

La eliminación remueve el nodo y todo su subárbol.

ID a eliminar

Eliminar

Buscar nodo

Si el nodo existe, se resalta en amarillo sobre React Flow.

ID a buscar

Buscar

Recorridos

Levelorder usa la Queue manual del backend.

Levelorder

Ver

Visualización React Flow

Raíz actual: Facultad de Ingeniería

Refrescar

Estado estructural

Representación JSON del árbol para validar la jerarquía académica sin depender de la vista.

```
{
  "height": 3,
  "levels": 4,
  "root": {
    "category": "faculty",
    "children": [
      {
        "category": "career",
        "children": [
          {
            "category": "cycle",
            "children": [
              {
                "category": "course",
                "children": [],
                "id": "math-1",
                "label": "Matemática I",
                "metadata": {
                  "credits": 5
                }
              }
            ]
          }
        ]
      }
    ]
  }
}
```

8.2 Árbol Binario BST

Objetivo

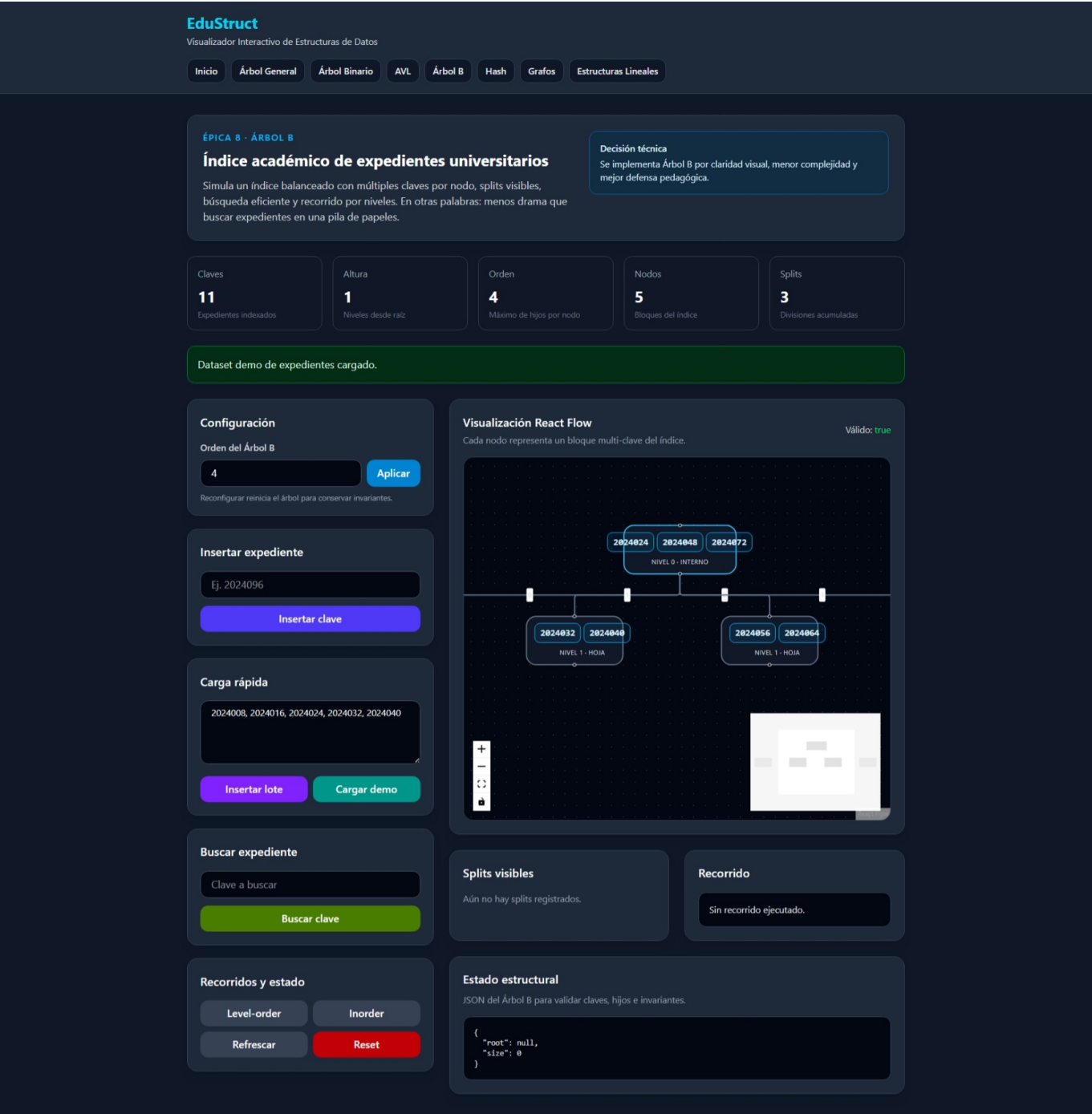
Demostrar inserciones ordenadas y recorridos clásicos en un árbol binario de búsqueda.

6 / 17

# Operaciones implementadas

- Insert
- Delete
- Search
- Inorder
- Preorder
- Postorder
- Levelorder

# Evidencia visual



## 8.3 Árbol AVL

## Objetivo

Visualizar balanceo automático mediante rotaciones LL, LR, RR y RL.

## Características destacadas

- Balance factor
- Alturas
- Rotaciones visibles
- Snapshots estructurales
- Recorridos animados

## Evidencia visual

EduStruct

Visualizador Interactivo de Estructuras de Datos

Inicio

Árbol General

Árbol Binario

AVL

Árbol B

Hash

Grafos

Estructuras Lineales

ÉPICA 7

Árbol AVL balanceado con rotaciones visibles

Cargar demo

Reiniciar

Inserta, elimina y busca IDs académicos en un árbol AVL. Cada operación actualiza altura, factor de balance y registra rotaciones LL, RR, LR o RL.

Estado del árbol AVL obtenido correctamente.

NODOS

7

ALTURA

2

NIVELES

3

BALANCE RAÍZ

0

ROTACIONES

1

BALANCEADO

Sí

Insertar ID

El AVL mantiene búsqueda O(log n) mediante balanceo automático.

ID, ej. 35

Insertar

Eliminar ID

Después de eliminar, el árbol recalcula alturas y rebalancea si hace falta.

ID a eliminar

Eliminar

Buscar ID

Si el ID existe, se resalta el nodo. En AVL el camino se mantiene corto.

ID a buscar

Buscar

Recorridos animados

Inorder confirma orden ascendente; levelorder expone niveles y balance.

Levelorder

Ejecutar

Velocidad Normal

Continuar

Reiniciar

Rotaciones registradas

Cada evento expone tipo, pivote y snapshots antes/después para explicar el rebalanceo.

LL

pivote 50

Visualización React Flow AVL

Raíz actual: 27 Última rotación: LL sobre 50

Refrescar

Limpiar marcas

Vista activa: Después de la operación

Estado estructural AVL

JSON del árbol para validar raíz, alturas, factores de balance e hijos.

```
{  "balanceFactor": 0,  "height": 2,  "isBalanced": true,  "levels": 3,  "root": {    "balanceFactor": 0,    "height": 3,    "isUnbalanced": false,    "left": {      "balanceFactor": 0,      "height": 2,      "isUnbalanced": false,      "left": {        "balanceFactor": 0,        "height": 1,        "isUnbalanced": false,        "left": null,        "right": null,        "value": 10      },      "right": {        "balanceFactor": 0,        "height": 1,        "isUnbalanced": false,        "left": null,        "right": null,        "value": 30      }    },    "right": {      "balanceFactor": 0,      "height": 1,      "isUnbalanced": false,      "left": null,      "right": null,      "value": 50    }  },  "right": {    "balanceFactor": 0,    "height": 1,    "isUnbalanced": false,    "left": null,    "right": null,    "value": 40  }  },  }
```

Última rotación

Snapshot técnico para explicar qué subárbol fue transformado.

```
{  "after": {    "balanceFactor": 0,    "height": 2,    "isUnbalanced": false,    "left": {      "balanceFactor": 0,      "height": 1,      "isUnbalanced": false,      "left": null,      "right": null,      "value": 30    },    "right": {      "balanceFactor": 0,      "height": 1,      "isUnbalanced": false,      "left": null,      "right": null,      "value": 50    }  },  "value": 40  },  }
```

## 8.4 Árbol B

### Objetivo

Simular índices académicos balanceados utilizando múltiples claves por nodo.

### Características implementadas

- Splits visibles
- Inserción balanceada

9 / 17

- Recorridos por nivel
- Validación estructural

Evidencia visual

EduStruct

Visualizador Interactivo de Estructuras de Datos

Inicio

Árbol General

Árbol Binario

AVL

Árbol B

Hash

Grafos

Estructuras Lineales

ÉPICA 6

Árbol Binario BST y recorridos

Inserta valores en un árbol binario de búsqueda, ejecuta recorridos clásicos y valida su representación visual con React Flow.

Cargar demo

Reiniciar

Estado del árbol binario obtenido correctamente.

NODOS

7

ALTURA

2

NIVELES

3

HOJAS

4

BALANCE RAÍZ

0

ARISTAS

6

Insertar valor

Los valores menores viajan a la izquierda y los mayores a la derecha.

Valor, ej. 55

Insertar

Eliminar valor

La eliminación mantiene la propiedad BST usando sucesor inorden.

Valor a eliminar

Eliminar

Buscar valor

Si el valor existe, se resalta en amarillo dentro del árbol.

Valor a buscar

Buscar

Recorridos animados

Inorder muestra los valores ordenados; levelorder usa la Queue manual. La animación revela cada nodo como si el árbol estuviera pasando lista.

Levelorder

Ejecutar

Velocidad Norm.

Continuar

Reiniciar

Visualización React Flow

Refrescar

Limpiar marcas

Raíz actual: 50

Estado estructural

Representación JSON del BST para validar raíz, hijos y métricas sin depender de la vista.

```
{  "height": 2,  "levels": 3,  "root": {    "left": {      "left": {        "left": null,        "right": null,        "value": 10      },      "right": {        "left": null,        "right": null,        "value": 40      },      "value": 25    },    "right": {      "left": {        "left": null,        "right": null,        "value": 40      },      "value": 50    },    "left": {      "left": null,      "right": null,      "value": 60    }  }  }
```

8.5 Tabla Hash

Objetivo

Implementar búsqueda académica mediante hashing y manejo de colisiones.

Estrategia utilizada

10 / 17

- ## Evidencia visual

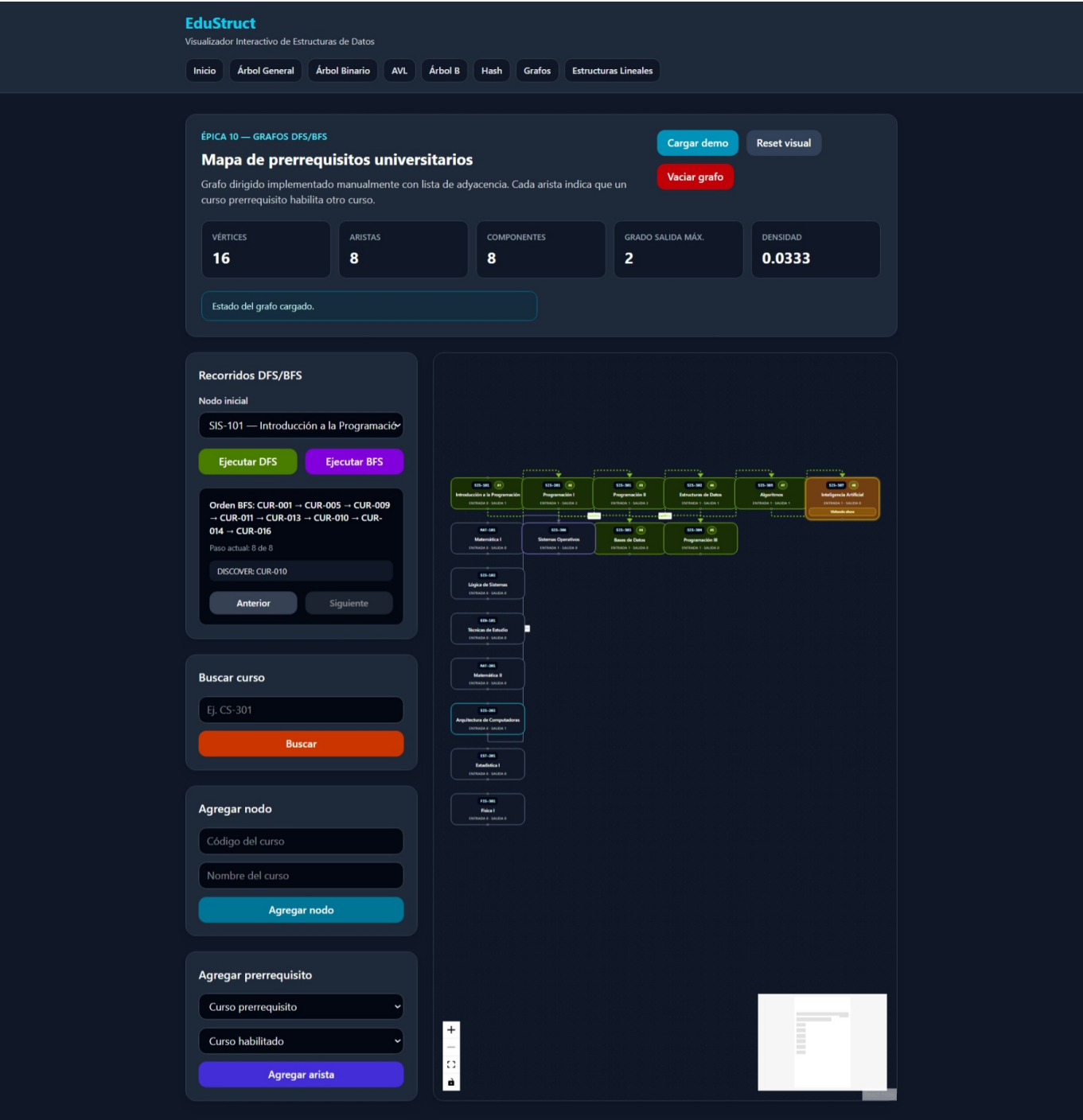


Representar prerrequisitos universitarios mediante grafos dirigidos.

# Algoritmos implementados

- DFS
- BFS
- Búsqueda de rutas
- Componentes
- Métricas del grafo

# Evidencia visual



# 8.7 Estructuras Lineales

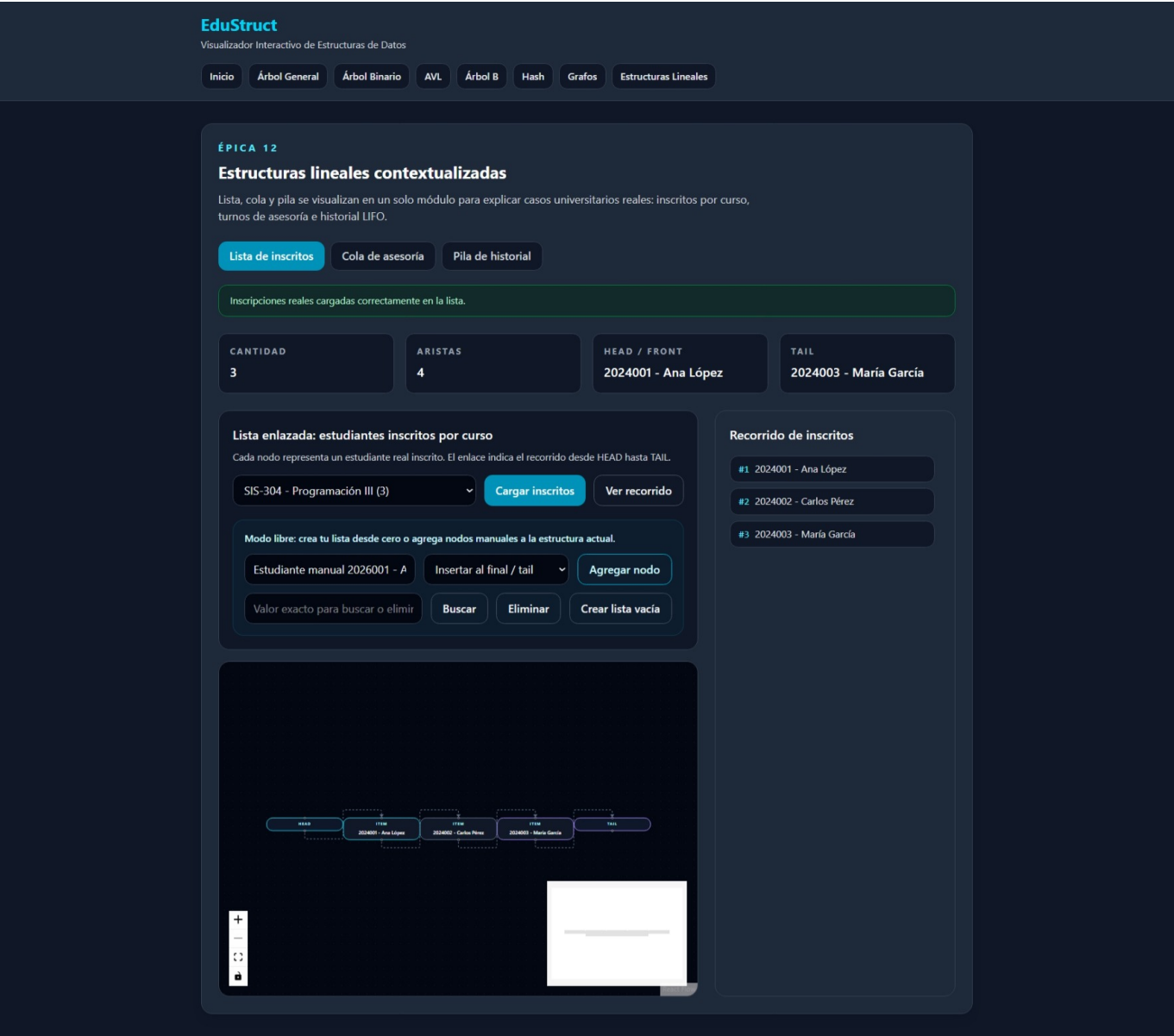
# Objetivo

Integrar lista, cola y pila en escenarios académicos reales.

# Casos implementados

| Estructura | Caso académico      |
|------------|---------------------|
| Lista      | Inscritos por curso |
| Cola       | Turnos de asesoría  |
| Pila       | Historial LIFO      |

# Evidencia visual



# 9. APIs REST

## Arquitectura API

El backend utiliza Flask con separación modular mediante blueprints y servicios especializados.

### Ejemplos de endpoints

| Método | Endpoint     | Descripción    |
|--------|--------------|----------------|
| GET    | /health      | Estado backend |
| POST   | /avl/insert  | Inserción AVL  |
| POST   | /bst/insert  | Inserción BST  |
| POST   | /hash/insert | Inserción hash |
| POST   | /graph/bfs   | Recorrido BFS  |

## 10. Visualización Interactiva

La visualización se realiza mediante React Flow, permitiendo:

- Zoom
- Pan
- Recorridos animados
- Renderizado dinámico
- Actualización reactiva
- Métricas visuales

## 11. Complejidad Algorítmica

| Estructura    | Inserción   | Búsqueda    | Eliminación |
|---------------|-------------|-------------|-------------|
| BST           | $O(h)$      | $O(h)$      | $O(h)$      |
| AVL           | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ |
| Árbol B       | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ |
| Hash          | $O(1)^*$    | $O(1)^*$    | $O(1)^*$    |
| Grafo DFS/BFS | $O(V+E)$    | $O(V+E)$    | —           |

\* Promedio esperado.

## 12. Dockerización y Entorno

El proyecto utiliza contenedores para desacoplar frontend y backend.

## Componentes

- Dockerfile backend
  - Dockerfile frontend
  - docker-compose
  - Redes internas
- 

## 13. Resultados Obtenidos

---

- Plataforma funcional desacoplada.
  - Visualización interactiva estable.
  - Implementación manual de estructuras.
  - Integración académica contextualizada.
  - Arquitectura escalable para futuras extensiones.
- 

## 14. Conclusiones

---

EduStruct permitió integrar teoría de estructuras de datos con un enfoque visual y práctico.

El proyecto demuestra:

- comprensión algorítmica,
- diseño modular,
- separación arquitectónica,
- visualización interactiva,
- aplicación contextualizada de estructuras clásicas.

Además, la implementación manual fortaleció la comprensión interna de algoritmos y recorridos.

---

## 15. Trabajo Futuro

---

### Mejoras posibles

- Persistencia en base de datos
  - Sistema multiusuario
  - Autenticación
  - Exportación de estructuras
  - Animaciones avanzadas
  - Métricas de rendimiento en tiempo real
  - Integración CI/CD
- 

## 16. Anexos

---

## Capturas del sistema

- Home
- Árbol General
- BST
- AVL
- Árbol B
- Hash
- Grafos
- Estructuras lineales

---

## Repositorio y estructura final

El proyecto EduStruct fue organizado utilizando una estructura monorepo con separación clara entre frontend, backend, documentación y datasets académicos.

Esta organización permitió desarrollar los módulos de manera desacoplada, facilitando el mantenimiento, la escalabilidad y el trabajo paralelo entre componentes visuales y lógicos.

```
/edustruct
├── backend
│   ├── app
│   │   ├── routes
│   │   ├── services
│   │   ├── structures
│   │   ├── schemas
│   │   └── serializers
│   └── Dockerfile
├── frontend
│   ├── src
│   │   ├── pages
│   │   ├── components
│   │   ├── services
│   │   └── layouts
│   ├── Dockerfile
│   └── vite.config.js
├── datasets
│   ├── carreras.json
│   ├── ciclos.json
│   └── cursos.json
├── docs
│   ├── final-report-pdf.md
│   └── images
├── docker-compose.yml
└── README.md
```

